POLYMARKET

V2 Smart Contracts Review

Security Assessment Report

Version: 2.0

July, 2026

Contents

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the Polymarket components in scope. The review focused solely on the security aspects of the Solidity implementation of the contracts, though general recommendations and informational comments are also provided.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the components in scope. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

In the course of providing this service, Sigma Prime may give views or opinions regarding the impact and risk of certain laws that may affect the project. However, Polymarket is put on notice that Sigma Prime is not a law firm, does not engage in the practice of law, and does not provide legal advice. Polymarket and readers should consult with their own legal counsel where relevant.

Document Structure

The first section provides an overview of the functionality of the Polymarket components contained within the scope of the security review. A summary followed by a detailed review of the discovered vulnerabilities is then given which assigns each vulnerability a severity rating (see Vulnerability Severity Classification), an *open/closed/resolved* status and a recommendation. Additionally, findings which do not have direct security implications (but are potentially of interest) are marked as *informational*.

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the Polymarket components in scope.

Overview

Polymarket Version 2 is a modular ERC-1155 based prediction market that replaces the existing Conditional Token Framework (CTF) based Polymarket contracts.

The new architecture supports three modules for different market types: binary markets, combinatorial markets and multiple outcome events. These markets make use of `pUSD`, the new collateral token which is backed by `USDC` or `USDC.e` tokens. Multiple chains are also supported with bridging of the `pUSD` collateral tokens, market positions or resolutions between supported chains.

Security Assessment Summary

Scope

The review was conducted on the files hosted on the Polymarket/polymarket-v2 repository.

The scope of this time-boxed review was strictly limited to files at commit c3be835.

Note: third party libraries and dependencies were excluded from the scope of this assessment.

Approach

The security assessment covered components written in Solidity.

The manual review focused on identifying issues associated with the business logic implementation of the contracts. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Ethereum Virtual Machine (for example, verifying correct storage/memory layout).

Additionally, the manual review process focused on identifying vulnerabilities related to known Solidity anti-patterns and attack vectors, such as re-entrancy, front-running, integer overflow/underflow and correct visibility specifiers.

To support the Solidity components of the review, the testing team may use the following automated testing tools:

- Aderyn: `https://github.com/Cyfrin/aderyn`
- Slither: `https://github.com/trailofbits/slither`
- Mythril: `https://github.com/ConsenSys/mythril`

Output for these automated tools is available upon request.

Coverage Limitations

Due to the time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified a total of 22 issues during this assessment. Categorised by their severity:

- Medium: 2 issues.
- Low: 10 issues.
- Informational: 10 issues.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the Polymarket components in scope. Each vulnerability has a severity classification which is determined from the likelihood and impact of each finding by the matrix given in the Appendix: Vulnerability Severity Classification.

A number of additional properties of the components, including optimisations, are also described in this section and are labelled as “informational”.

Each vulnerability is also assigned a **status**:

- ***Open:*** the finding has not been addressed by the project team.
- ***Resolved:*** the finding was acknowledged by the project team and updates to the affected components have been made to mitigate the related risk.
- ***Closed:*** the project team has reviewed and acknowledged the finding, but the recommended remediation has not been implemented. The project team has typically provided a documented rationale supporting this decision, including the approach taken and any associated risk considerations.

Summary of Findings

ID	Description	Severity	Status
POLY1-01	Buggy Arbitrator Can Block Emergency Admin Resolution And Dispute Escalation	Medium	Resolved
POLY1-02	Safe Wallet Order Validation Can Bypass Multisig Controls	Medium	Closed
POLY1-03	`compress()` Can Burn YES Position And Return Neither Collateral Nor A New Position	Low	Closed
POLY1-04	Disputer Can Dispute Multiple Times	Low	Resolved
POLY1-05	Floor Rounding In `_calculateTakingAmount` Allows An Operator To Steal Seller Positions	Low	Closed
POLY1-06	Bridge Lacks Admin Liquidity Rebalance Feature	Low	Closed
POLY1-07	Auto-Derived Fallback Condition Bypasses Resolution Pause	Low	Closed
POLY1-08	`forceRerequest` Drops Proposer Reward, Weakening The Recovery Path	Low	Resolved
POLY1-09	Gas Griefing Of Batched `migratePositions` Calls	Low	Resolved
POLY1-10	Gas-Grief Of Operators By Invalidating Transfers Before Matching Orders	Low	Closed
POLY1-11	`OracleAggregator` Dispute Window Continues Running While Paused	Low	Closed
POLY1-12	`OracleAggregator` Does Not Enforce Majority Reporter Quorum	Low	Closed
POLY1-13	`EOAReporterModule` Singleton Claim Is Inconsistent With Aggregator Voting Model	Informational	Resolved
POLY1-14	Incomplete Validation In Oracle Aggregator's `initializeRequest()`	Informational	Closed
POLY1-15	Missing Upper Bound On `resultLength`	Informational	Closed
POLY1-16	Oracle Request's Arbitrator Can Resolve Before Arbitration Is Triggered	Informational	Closed
POLY1-17	Order Remaining Amount Truncated To 248 Bits	Informational	Closed
POLY1-18	Orders Have No Expiry	Informational	Closed
POLY1-19	`pUSD` Lacks Fine-Grained User Controls For Compliance Responses	Informational	Closed
POLY1-20	`removeBridge()` Can Strand In-Flight CCIP Bridge Transfers	Informational	Closed
POLY1-21	Reserved Bits In Structured IDs Are Not Validated	Informational	Closed
POLY1-22	Miscellaneous General Comments	Informational	Closed

POLY1-01	Buggy Arbitrator Can Block Emergency Admin Resolution And Dispute Escalation		
Assets	`src/oracle/OracleAggregator.sol`		
Status	**Resolved:** See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

A buggy or malicious Arbitrator can block execution of `OracleAggregator.resolveResult()`, which admins (and Arbitrators themselves) use to manually resolve a request. Arbitrators have multiple mechanisms to block this path.

`resolveResult()` calls `onArbitrationResolved()` on the Arbitrator using a best-effort `try/catch` block:

`src/oracle/OracleAggregator.sol::resolveResult()`

```
// Admin override during arbitration: notify the arbitrator so it can clear its local
// `isActive` flag. Best-effort: a revert from the hook (paused arbitrator, buggy
// implementation, etc.) must not block emergency admin resolution.
if (!isArb && state.status == ResolutionStatus.ArbitrationRequested) {
    try IArbitratorModule(cfg.arbitratorModule).onArbitrationResolved(_requestId) { }
    catch {
        emit ArbitratorHookFailed(_requestId, cfg.arbitratorModule);
    }
}
```

The intent is clear from the comments and interface docs: the hook *must not* block emergency admin resolution, and is only used on a best-effort basis.

However, using a plain high-level `try/catch` external call is not sufficient to guarantee that property, and the following mechanisms can be abused by a buggy or malicious Arbitrator to block further execution of `resolveResult()`.

Gas exhaustion:

The call effectively forwards all remaining gas to the Arbitrator. Although the EVM reserves `1/64` of the call's gas for the caller under EIP-150 semantics, that residual gas may still be insufficient for the rest of `resolveResult()`, which performs additional state writes and downstream resolution calls after the hook returns.

Return-bomb style griefing:

A malicious Arbitrator can return or revert with extremely large return data, forcing costly return data handling/copying and potentially causing the caller to run out of gas before it reaches the catch path or before it finishes the rest of `resolveResult()`.

EOA / non-contract Arbitrator addresses:

Solidity verifies that a contract contains code via `EXTCODESIZE` before performing a call, and reverts if the target has no code. This revert is *not* caught by a high-level `try/catch` block, which is only intended to respond to reverts resulting from an actual invocation of the target contract. Consequently, if `cfg.arbitratorModule` has no code, the call will revert and block emergency admin resolution.

Reentrancy:

A malicious arbitrator can also cause an admin `resolveResult()` call to revert by reentering `resolveResult()` with a different `_result` for the same `_requestId` during the `onArbitrationResolved()` hook invocation. Because the hook is

executed before the outer call finalises the request, the arbitrator can set an incompatible payout for the same request. When the original outer call later proceeds into `_finalizeConditions()`, it encounters an already-resolved target state with a conflicting payout and reverts with `ExistingPayoutMismatch`. As a result, the effects of the arbitrator hook escape the try/catch block and can cause the outer call to revert, blocking emergency admin resolution.

Similar blocking in dispute escalation:

The function `disputeResult()` also calls an arbitrator hook, where a malicious or buggy arbitrator can revert when `onArbitrationTriggered()` is called to block dispute escalation. In contrast with `resolveResult()`, this function does not use a `try/catch` block, so any revert from the hook will directly block dispute escalation. However, disputes are resolved by the arbitrator, so this is less of a concern.

src/oracle/OracleAggregator.sol::disputeResult()

```
if (state.disputeCount >= cfg.disputerThreshold) {
    state.status = ResolutionStatus.ArbitrationRequested;

    emit ArbitrationTriggered(_requestId);

    // @audit: Can revert and block dispute escalation
    IArbitratorModule(cfg.arbitratorModule).onArbitrationTriggered(_requestId, state.proposedResultHash);
}
```

This issue has a high impact as it defeats the documented best-effort guarantee of the emergency admin resolution path. A misbehaving Arbitrator can prevent admins from manually resolving a request during arbitration, which is precisely the scenario this code path was designed to handle, and can leave funds stuck in an unresolvable request.

This issue has a low likelihood as it requires the configured Arbitrator module to be either buggy or actively malicious. Arbitrator modules are trusted, privileged components, so the failure mode depends on either a bug in a deployed Arbitrator or a compromised/malicious Arbitrator deliberately sabotaging the override path.

Recommendations

Replace the high-level `try/catch` hook with a tightly controlled low-level call and:

- Verify the arbitrator has code
- Limit gas forwarded to the hook
- Discard `return data`
- Add reentrancy protection

Resolution

A partial fix has been implemented in commit e80782. The development team removed the `try/catch` block in `resolveResult()` and made use of a low level call. This low level call now ignores return data and ensures that EOA Arbitrator addresses cannot interrupt the resolution.

However, two raised recommendations remain unresolved. The call still forwards 63/64 of the call's gas to the external call and no reentrancy protection has been added to `resolveResult()`.

POLY1-02	Safe Wallet Order Validation Can Bypass Multisig Controls		
Assets	`src/exchange/Exchange.sol`		
Status	**Closed:** See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

The exchange does not properly validate Safe and proxy wallet orders.

For `POLY_PROXY` and `POLY_GNOSIS_SAFE` orders, the exchange mainly checks two things: that the order was signed by the original EOA, and that the maker wallet address matches the address expected from that EOA.

The problem is that this does not check the wallet's current security settings.

A Safe wallet may later be changed into a multisig wallet. The original EOA may also be removed as an owner. Even after these changes, the exchange may still accept an order signed only by that old EOA. It does not check whether the signer is still part of the Safe, or whether the current multisig threshold has been met.

The weak validation is:

`src/exchange/Exchange.sol::Exchange::_isValidSignature()`

```
function _isValidSignature(bytes32 orderHash, Order calldata order) internal view returns (bool) {
    //....
    if (order.signatureType == SignatureType.POLY_PROXY) {
        return _isValidEOASignature(orderHash, order.signer, order.signature)
            && _getProxyWalletAddress(order.signer) == order.maker;
    }
    if (order.signatureType == SignatureType.POLY_GNOSIS_SAFE) {
        return _isValidEOASignature(orderHash, order.signer, order.signature)
            && _getSafeWalletAddress(order.signer) == order.maker;
    }
    return false;
}
```

`src/exchange/Exchange.sol::Exchange::_getSafeWalletAddress()`

```
function _getSafeWalletAddress(address signer) internal view returns (address) {
    return _computeCreate2Address(SAFE_FACTORY, SAFE_BYTECODE_HASH, _safeSalt(signer));
}

function _safeSalt(address signer) internal pure returns (bytes32 salt) {
    assembly ("memory-safe") {
        mstore(0x00, signer)
        salt := keccak256(0x00, 0x20)
    }
}
```

This only confirms that the EOA signed the order; and the Safe address matches the address linked to that EOA.

It does not confirm that the EOA is still an owner of the Safe. It also does not confirm that the Safe's current multisig threshold has approved the order.

When a Safe wallet is created, `SafeProxyFactory.createProxy()` is called. The `salt` used to create the wallet is based on the owner address. This means the wallet address is permanently linked to the original owner address.

packages/safe-factory/contracts/SafeProxyFactory.sol::SafeProxyFactory

```
function createProxy(
    address paymentToken,
    uint256 payment,
    address payable paymentReceiver,
    Sig calldata createSig
)
    external
{
    address owner = _getSigner(paymentToken, payment, paymentReceiver, createSig);

    GnosisSafe proxy;
    bytes memory deploymentData = getContractBytecode();
    bytes32 salt = getSalt(owner);

    assembly {
        proxy := create2(0x0, add(0x20, deploymentData), mload(deploymentData), salt)
    }
    require(address(proxy) != address(0), "create2 call failed");

    {
        address[] memory owners = new address[](1);
        owners[0] = owner;
        proxy.setup(owners, 1, address(0), "", fallbackHandler, paymentToken, payment, paymentReceiver);
    }

    emit ProxyCreation(proxy, owner);
}
```

Assume, Bob creates a Safe wallet from his EOA, so Bob is the first owner of the Safe.

Later, Bob adds more owners and increases the signature threshold. At this point, the wallet is meant to work as a proper multisig wallet. The Safe also approves tokens for the exchange so it can trade on Polymarket.

Bob can still sign an order on behalf of the Safe by using only his EOA. The exchange may accept that order because it only checks that Bob's EOA signed it and that the Safe address matches the deterministic address linked to Bob.

This bypasses the multisig threshold.

The issue is even worse if Bob is later removed from the Safe. Bob may still be able to sign an order, and the exchange may still accept it because the EOA signature is valid and the Safe address still matches the address originally linked to Bob.

Once an operator fills the order, the exchange can move approved tokens from the Safe. This can happen even though the Safe's current owners did not approve the trade.

This issue has a high impact as a removed or no-longer-authorised signer can unilaterally cause the Safe's approved tokens to be moved by the exchange, bypassing the multisig threshold and the wallet's current authorisation rules.

This issue has a low likelihood as it requires the Safe to have been reconfigured after creation, either by adding owners and raising the threshold or by removing the original deployer, while still keeping the exchange approvals in place. Polymarket-deployed Safes are not expected to be reconfigured this way in normal use.

Recommendations

Modify the implementation such that the exchange does not rely only on the original EOA signature and the deterministic wallet address.

For Safe and proxy wallets, the exchange should check the wallet's current authorisation rules before accepting an order.

Resolution

The development team have stated that generic Safe Wallets are not intended for use with the Polymarket V2 system:

> "*Safe wallets are only allowed from Polymarket's own safe factory. We don't support wallets where users rotate owners on the safe or add new owners/threshold.* "

POLY1-03	`compress()` Can Burn YES Position And Return Neither Collateral Nor A New Position		
Assets	`src/modules/CombinatorialModule.sol`		
Status	**Closed:** See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

In `CombinatorialModule.compress()`, when compressing a YES combinatorial position (`outcomeIndex == 0`), the function computes the post-compression amount as:

`src/modules/CombinatorialModule.sol::CombinatorialModule::compress()`

```
if (outcomeIndex == 0) {
    positionAmount = FixedPointMathLib.mulDiv(_amount, payoutFactor, PAYOUT_FACTOR_DENOMINATOR);
    if (positionAmount != 0) {
        if (remainingCount == 0) {
            collateralOut = positionAmount;
            positionAmount = 0;
        } else {
            PositionId[] memory trimmed = _trimArray(remaining, remainingCount);
            newPositionId = _storeLegsFromMemory(trimmed).computePositionId(0);
        }
    }
}
```

If `positionAmount == 0`, the function mints neither new collateral nor new position tokens, but still burns the original position unconditionally:

`src/modules/CombinatorialModule.sol::CombinatorialModule::compress()`

```
if (collateralOut != 0) COLLATERAL_TOKEN.mint(_to, collateralOut);
if (PositionId.unwrap(newPositionId) != 0) POSITION_MANAGER.mint(_to, newPositionId, positionAmount);

POSITION_MANAGER.burn(_positionId, _amount);
```

Consequently, if a YES position has a nonzero but sub-unit compressed value, `compress()` rounds the amount down to zero, mints nothing, and burns the entire original position.

The root cause is that for YES positions, `positionAmount == 0` is treated as "mint nothing," but there is no guard preventing the original position from being burned anyway. The function silently destroys the user's remaining claim when the compressed YES output is below one token unit.

Example

Assume a user holds 9,999 `YES(A & B & C)` with the following leg resolutions:

- `A` resolved as $[1\%, 99\%]$
- `B` resolved as $[1\%, 99\%]$
- `C` unresolved

For the stored YES legs:

- $A_{yes} = 1\%$
- $B_{yes} = 1\%$

So the resolved payout factor is $0.01 \times 0.01 = 0.0001$. Since `payoutFactor` is represented as a fixed-point decimal with 36 decimals, `payoutFactor` $== 10^{32}$.

Then `compress()` computes the position amount as: $9999 \times 0.0001 = 0.9999$, which rounds down to zero. More precisely, when considering the fixed-point scaling, the function computes `positionAmount` as $\lfloor 9999 \times 10^{32} / 10^{36} \rfloor = \lfloor 0.9999 \rfloor = 0$.

Since `C` is still unresolved, `remainingCount > 0`, indicating that the function should create a new compressed position for the remaining unresolved leg.

src/modules/CombinatorialModule.sol::CombinatorialModule::compress()

```
(bool resolved, uint256 conditionPayout) = _getConditionPayout(storedLegs[i]);
if (!resolved) {
    remaining[remainingCount] = storedLegs[i];
    ++remainingCount;
}
// ...
```

But because `positionAmount == 0`, the YES branch never sets `newPositionId` for the replacement position, so no new position is minted. A replacement `YES(C)` position is not minted, nor is any collateral.

Finally, the function still burns the original `9999 YES(A & B & C)`.

So the result is:

- **burned:** 9,999 `YES(A & B & C)`
- **minted:** nothing

This is a complete loss of the user's position due solely to floor-rounding during compression.

A user calling `compress()` may reasonably expect one of two outcomes:

1. To receive collateral for the resolved portion, or
2. receive a smaller new position for the unresolved remainder.

But in this edge case, they receive neither, the protocol simply deletes the position.

Even if the remaining value is small, silently burning the entire user position is dangerous behaviour and can lead to irreversible loss.

This issue has a medium impact as it results in the irreversible loss of a user's entire combinatorial YES position without any compensating mint of collateral or replacement position.

This issue has a low likelihood as it requires a specific combination of circumstances: the user must hold a YES combinatorial position whose resolved legs drive the payout factor low enough that it rounds down to zero, while at least one leg remains unresolved. Users will typically hold position amounts large enough to avoid this rounding edge, so triggering the loss requires either small holdings or extreme resolved payouts on multiple legs.

Recommendations

Add an explicit guard for the YES branch so that the function does not burn the original position when compression would produce zero output due to rounding.

```
if (outcomeIndex == 0) {
    positionAmount = FixedPointMathLib.mulDiv(_amount, payoutFactor, PAYOUT_FACTOR_DENOMINATOR);
    if (positionAmount != 0) {
        if (remainingCount == 0) {
            // All legs resolved: the YES payout is pure collateral.
            // Move the calculated value into collateralOut and clear
            // positionAmount so the Compressed event correctly reports
            // zero for the minted position amount.
            collateralOut = positionAmount;
            positionAmount = 0;
        } else {
            PositionId[] memory trimmed = _trimArray(remaining, remainingCount);
            newPositionId = _storeLegsFromMemory(trimmed).computePositionId(0);
        }
    } else if(payoutFactor == 0) {
        revert("positionAmount is rounded to zero");
    }
}
```

More generally, consider requiring callers to provide a minimum acceptable output.

Resolution

The development team have acknowledged this issue and closed it with the following comment:

> *"[This issue affects] Dust-level amounts only."*

POLY1-04	Disputer Can Dispute Multiple Times		
Assets	`src/oracle/OracleAggregator.sol`		
Status	**Resolved:** See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

A single disputer module can force arbitration by repeatedly calling `disputeResult()`, bypassing the intended threshold consensus mechanism.

The `disputeResult()` function increments `disputeCount` each time it is called by a registered disputer module, but unlike `reportResult()`, it does not track whether a specific module has already disputed:

`src/oracle/OracleAggregator.sol::disputeResult()`

```
function disputeResult(bytes32 _requestId) external whenUnpaused {
    (EventId eventId, RequestConfig storage cfg) = _getRequestConfigForRequestId(_requestId);

    ResolutionState storage state = resolutionStates[_requestId];
    require(_isActiveStatus(state.status), RequestNotActive());
    require(state.proposedResultHash != bytes32(0), NoProposalToChallenge());
    require(block.timestamp < state.disputeWindowEnd, DisputeWindowExpired());

    require(isDisputerModule[eventId][msg.sender], NotRegisteredModule());

    state.disputeCount++; // @audit No check if this module has already disputed

    emit ResultDisputed(_requestId, msg.sender);

    if (state.disputeCount >= cfg.disputerThreshold) {
        state.status = ResolutionStatus.ArbitrationRequested;
        // ...
    }
}
```

In contrast, `reportResult()` prevents double-voting through a dedicated mapping:

`src/oracle/OracleAggregator.sol::reportResult()`

```
require(!hasReporterVoted[_requestId][msg.sender], AlreadyVoted());
hasReporterVoted[_requestId][msg.sender] = true;
```

A malicious or buggy disputer module can call `disputeResult()` repeatedly within the dispute window to single-handedly reach the `disputerThreshold` and trigger arbitration. This undermines the security model where multiple independent disputers should agree before escalating to costly arbitration.

This issue has a medium impact as it allows a single disputer to force arbitration, potentially causing unnecessary operational costs and disrupting the intended consensus mechanism, but does not directly result in loss of funds.

This issue has a low likelihood as it requires the attacker to be a registered disputer module, which is set by operators during request initialisation. However, the exploit is trivial once registered, and the system should enforce the threshold at the aggregator level rather than relying on disputer implementations to self-limit.

Recommendations

Add a mechanism that tracks whether each disputer module has already disputed a specific request, similar to the existing `hasReporterVoted` pattern.

Resolution

The finding has been resolved in commit e80782, where a check was added to `disputeResult()` to track if a reporter has already voted and revert if this occurs.

POLY1-05	Floor Rounding In `_calculateTakingAmount` Allows An Operator To Steal Seller Positions		
Assets	`src/exchange/Exchange.sol`		
Status	**Closed:** See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

A malicious or compromised operator can fill orders such that the maker's amount received rounds to zero, transferring the maker's assets (position tokens or collateral) to the taker for free. The Exchange's price-enforcement and accounting checks all reduce to comparisons against the same floored quantity, so they are all satisfied at zero and the trade settles.

The Exchange computes payments using the helper `_calculateTakingAmount()` in `Exchange.sol`:

`src/exchange/Exchange.sol::_calculateTakingAmount()`

```
/// @dev Computes the taking amount from the making amount and ratio.
function _calculateTakingAmount(uint256 makingAmount, uint256 makerAmount, uint256 takerAmount)
    internal
    pure
    returns (uint256)
{
    return makingAmount * takerAmount / makerAmount; // @audit floors to zero when makingAmount * takerAmount < makerAmount
}
```

The function effectively scales `makingAmount` by the price `takerAmount / makerAmount`. However, when `makingAmount * takerAmount < makerAmount`, the result rounds down to zero.

Consider the following example:

- Carla submits a SELL order with `makerAmount = 1,000,000` and `takerAmount = 1,001,000,001`.
- Bob submits a BUY order with `makerAmount = 50,000,000` and `takerAmount = 50,000,000`.

The following steps result in Bob receiving Carla's position tokens for free:

1. The operator matches them, and invokes `matchOrders()` with Bob's order as the `takerOrder`, Carla's order as `makerOrders[0]`, and `fillAmounts[0] = 1001`.
2. After validation, execution continues to `_executeComplementaryBuyFastPath()`.
3. `_validateAndUpdate()` ensures that Carla's order can support filling 1001 tokens, and reduces the order's remaining amounts accordingly, but does not check the price or the resulting payment.
4. The `taking` value is computed by `_calculateTakingAmount()`, which returns `0` as described above.
5. 1001 position tokens are transferred from Carla to Bob.
6. Bob's payment is computed as `taking - fee`, which is zero, so no collateral is transferred from Bob to Carla.
7. The loop over all maker orders exits, some final checks are performed to ensure that Bob received the expected amount of position tokens, and the function returns successfully.

As a result, Carla loses 1001 position tokens and Bob receives them having transferred zero collateral.
Note that although in the illustrative example Bob places a BUY order, the issue also occurs if Bob places a SELL order.

This issue has a medium impact as it allows the operator to drain assets from any maker whose order is priced such that floor rounding can be triggered. However, the amount of lost funds is bounded by the price ratio and size of the maker's order.

This issue has a low likelihood as it requires a malicious or compromised `OPERATOR_ROLE` and a maker's order whose `makerAmount` exceeds `takerAmount`. The latter is the default for any market priced below `1` collateral unit per share, which should generally be the case for most prediction markets. However, it is unlikely that an operator would intentionally craft such an order fill, it is easily detectable onchain, and in practice gas costs may eclipse the value of the (bounded) stolen position.

Recommendations

Reject small order fills that would result in zero-value payments.

Resolution

The development team have stated this is known behaviour.

POLY1-06	Bridge Lacks Admin Liquidity Rebalance Feature		

Assets	`src/bridge/abstract/BridgeBase.sol`
	`src/bridge/CcipBridge.sol`
	`src/collateral/CollateralToken.sol`
	`src/collateral/CollateralOnramp.sol`
	`src/collateral/CollateralOfframp.sol`		

Status	**Closed:** See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Medium

Description

The end-to-end "wrap → bridge → unwrap" path effectively locks `USDC` (or `USDC.e`) on the source chain and unlocks it on the destination chain, but neither the bridge or the collateral system exposes an administrative mechanism for moving the underlying `USDC` between chains. Persistent liquidity asymmetry between chains can leave `pUSD` holders on a low-liquidity chain unable to unwrap to `USDC` even though the bridge is functioning normally.

The mechanics that produce this exposure are:

- The bridge itself is a burn-and-mint design over `pUSD`. `BridgeBase.bridgeCollateral()` burns `pUSD` from the sender on the source chain, and `BridgeBase._handleCollateralReceive()` mints `pUSD` to the recipient on the destination chain. The cross-chain message carries no underlying value.
- `pUSD` is a wrapper around `USDC` and `USDC.e` . `CollateralToken.wrap()` mints `pUSD` and transfers the underlying asset to an external `VAULT`. `CollateralToken.unwrap()` performs the inverse, pulling the underlying asset from `VAULT` via `safeTransferFrom` and burning the `pUSD`.
- The user-facing flow is therefore: deposit `USDC` into the source-chain vault via `wrap()`, bridge `pUSD` to the destination chain, then `unwrap()` against the destination-chain vault. The destination-chain vault must hold sufficient `USDC` for the unwrap to succeed.

This issue has a low impact as the failure mode is service disruption rather than loss of funds: `pUSD` holders retain a valid claim, but cannot exit to `USDC` on the affected chain until liquidity is refilled. However, affected users may self-mitigate by bridging their `pUSD` to a chain with sufficient liquidity, where the unwrap can occur. If supported by the vault, the development team can rebalance underlying `USDC` manually (for example via Circle's CCTP) and refill the vault.

This issue has a medium likelihood as cross-chain liquidity drift is a near-certainty over the lifetime of a multi-chain bridge of this shape rather than a corner case; the imbalance does not need an attacker to manifest.

Recommendations

Ensure that the vault and/or collateral system has a liquidity management feature which allows underlying `USDC` to be moved between chains as part of normal operations.

Alternatively, Chainlink CCIP supports transferring `USDC` via a mint/burn mechanism across several chains. Consider adding functionality to the bridge to support liquidity rebalancing by administrators.

Resolution

The development team have acknowledged this issue and will consider adding this feature in the future.

POLY1-07	Auto-Derived Fallback Condition Bypasses Resolution Pause		
Assets	src/modules/NegRiskModule.sol src/modules/abstract/OracleModule.sol		
Status	**Closed:** See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

`NegRiskModule._finalizeNegriskResolution()` auto-resolves the synthetic Other (fallback) condition without consulting `resolutionPausedAt`, allowing an admin's per-condition resolution pause on the fallback to be silently bypassed when the last real condition in an event is reported.

Per-condition resolution pausing is an explicit admin control surface in `OracleModule`. Both `pauseResolution()` and the `onlyResolver` modifier confirm that an admin is expected to be able to pause resolution for any single `ConditionId`:

src/modules/abstract/OracleModule.sol::OracleModule::onlyResolver()

```
modifier onlyResolver(ConditionId _id) {
    require(hasAnyRole(msg.sender, BRIDGE_ROLE | RESOLVER_ROLE), Unauthorized());
    if (hasAllRoles(msg.sender, RESOLVER_ROLE)) {
        require(_id.resolutionChain() == uint256(_resolutionChain()), InvalidResolutionChain());
    }
    require(resolverPausedAt[msg.sender] == 0, ResolverIsPaused());
    require(resolutionPausedAt[_id] == 0, ResolutionIsPaused());
    _;
}
```

The three resolution entry points that ordinarily reach the Other condition all honour this pause: `reportResult()` via the `onlyResolver` modifier, `resolveConditionToNo()` via an inline `resolutionPausedAt` check, and the bridge path which also passes through `onlyResolver`. The auto-derivation branch inside `_finalizeNegriskResolution()` does not:

src/modules/NegRiskModule.sol::NegRiskModule::_finalizeNegriskResolution()

```
function _finalizeNegriskResolution(ConditionId _conditionId, uint256[] memory _result) internal {
    // ... snip: real-condition storage and accounting ...

    // Resolve fallback if we can
    ConditionId fallbackConditionId = eventId_.computeConditionId(conditionCount_);
    if (
        (conditionsResolved_ == conditionCount_ || resultsSum_ == RESULT_DENOMINATOR)
            && result[fallbackConditionId].length == 0
    ) {
        // @audit no `resolutionPausedAt[fallbackConditionId] == 0` check
        uint256[] memory fallbackResult = new uint256[](2);
        fallbackResult[0] = RESULT_DENOMINATOR - resultsSum_;
        fallbackResult[1] = resultsSum_;

        _storeResult(fallbackConditionId, fallbackResult);

        // ... snip
    }
    // ... snip
}
```

If an admin invokes `pauseResolution()` on the fallback condition alone, a subsequent resolver report on the last real condition in the event will trigger the auto-derivation branch and store the fallback result regardless of the pause.

This issue has a low impact as the derived payout is mathematically determined from the already-stored real-condition results, so it does not produce an incorrect outcome; only the redemption-halt aspect of the pause is undermined.

This issue has a low likelihood as triggering it requires the admin to have selected the fallback for pause and a resolver to subsequently complete the event.

Recommendations

In `_finalizeNegriskResolution()`, skip the auto-derivation branch when `resolutionPausedAt[fallbackConditionId]` is non-zero. This will also require a pathway to resolving the fallback to YES once the pause is lifted (NO can be resolved via the existing `resolveConditionToNo()`).

Resolution

The development team have closed this issue with the following comment:

> "*The fallback condition is never paused by itself; pausing the real conditions also serves as a pause on the fallback.*"

POLY1-08	`forceRerequest` Drops Proposer Reward, Weakening The Recovery Path		
Assets	`src/oracle/modules/OptimisticOracleModule.sol`		
Status	**Resolved:** See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The admin escape hatch for stuck Optimistic Oracle requests recreates the replacement request with zero reward, weakening the recovery path in the exact scenario it is meant to handle.

`OptimisticOracleModule.forceRerequest()` is intended to serve as an admin escape hatch for stuck requests:

src/oracle/modules/OptimisticOracleModule.sol::OptimisticOracleModule::forceRerequest()

```
/// @notice Force a re-request for a stuck condition.
/// @dev Admin escape hatch for when settle() reverts due to
///      an invalid price from the QuorumOracle / upstream.
function forceRerequest(
    bytes32 _conditionId,
    bytes calldata _ancillaryData,
    address _bondToken,
    uint256 _bondAmount,
    uint256 _liveness
) external onlyAdmin {
    ConditionId conditionId = ConditionIdLib.from(_conditionId);
    if (currentRequestId[conditionId] == bytes32(0)) revert RequestNotInitialized();

    (bytes32 identifier,) = _determineIdentifierAndOutcomeCount(conditionId);
    _reRequest({
        _conditionId: conditionId,
        _identifier: identifier,
        _ancillaryData: _ancillaryData,
        _bondToken: _bondToken,
        _bondAmount: _bondAmount,
        _reward: 0,
        _liveness: _liveness
    });
}
```

However, the function always creates the replacement request with `_reward: 0`.

This is problematic because the module's normal reward-recycling design depends on the reward being refunded only when a dispute occurs, not merely because the request is stuck or settlement fails.

The normal dispute path explicitly assumes reward recycling:

src/oracle/modules/OptimisticOracleModule.sol::OptimisticOracleModule::priceDisputed()

```
/// @notice Called when a price is disputed
/// @dev Re-requests immediately. Reward is recycled via
///      refundOnDispute (setEventBased).
///      Because of the way callbacks are set, this function
///      will only be called for the "first" request, and not
///      for subsequent re-requests.
```

It then reuses the tracked reward on automatic re-request:

src/oracle/modules/OptimisticOracleModule.sol::OptimisticOracleModule::priceDisputed()

```
_reRequest({
    _conditionId: conditionId,
    _identifier: identifier,
    _ancillaryData: ancillaryData,
    _bondToken: req.currency,
    _bondAmount: req.requestSettings.bond,
    _reward: currentReward[conditionId],
    _liveness: req.requestSettings.customLiveness
});
```

The module stores this reward specifically for reuse:

src/oracle/modules/OptimisticOracleModule.sol::OptimisticOracleModule

```
/// @dev MOOv2 zeroes request.reward on dispute, so the module
///      tracks the active reward per condition for re-requests.
mapping(ConditionId => uint256) internal currentReward;
```

But the `OptimisticOracleV2` implementation shows that reward refund occurs only in the dispute path:

UMAprotocol/protocol/.../implementation/OptimisticOracleV2.sol::OptimisticOracleV2::disputePriceFor()

```
// Compute refund.
uint256 refund = 0;
if (request.reward > 0 && request.requestSettings.refundOnDispute) {
    refund = request.reward;
    request.reward = 0;
    request.currency.safeTransfer(requester, refund);
}
```

That logic is inside `disputePriceFor()`, not settlement. So the original request becomes stuck if: settlement reverts, the upstream/quorum oracle returns an invalid value or some other non-dispute failure occurs.

The original reward then may not have been refunded back to the module. As a result, `forceRerequest()` can create a replacement request with no recycled reward and no newly funded reward.

The new OO request is still technically valid, but it may be economically unattractive because there is no requester-funded proposer incentive.

The function is presented as a recovery mechanism for stuck requests, but it weakens the economics exactly in the scenario where the normal automated flow already failed.

If proposers depend on the reward to participate, a zero-reward recovery request may remain unresolved or take much longer to resolve. This undermines the effectiveness of the “admin escape hatch” as a practical liveness recovery tool.

For example:

1. Operator creates the original request with reward = 100 USDC.
2. The request later becomes stuck because `settle()` reverts due to invalid upstream data.
3. No dispute occurs, so the refund-on-dispute path is never triggered.
4. The 100 USDC reward is therefore not returned to the module through dispute refund logic.
5. Admin calls `forceRerequest()`.
6. The module creates a replacement OO request with `_reward: 0`.

This results in three problems:

- The original reward may still be tied to the old request lifecycle.
- The replacement request offers no reward to proposers.
- The recovery flow may become economically unattractive or ineffective.

This issue has a low impact as it does not result in loss of funds or direct corruption of state, but it does undermine the liveness guarantee of the documented admin recovery path. A zero-reward replacement request may remain unresolved for an extended period or fail to attract any proposer at all, prolonging the stuck state of the original condition.

This issue has a low likelihood as it requires the original request to have become stuck through a non-dispute failure mode (such as `settle()` reverting on invalid upstream data) so that the original reward was never refunded, and then for the admin to invoke `forceRerequest()` expecting normal recovery economics. The combination of upstream failure plus zero unfunded recovery is necessary to expose the issue.

Recommendations

Ensure `forceRerequest()` does not blindly zero out the reward.

Instead, the implementation should explicitly account for whether the original reward has been refunded, remains locked in the old request lifecycle, or has been replaced by fresh admin funding.

If the replacement request is intended to include a reward but the module cannot fund it, the function should revert rather than silently creating an economically unattractive zero-reward request.

Resolution

This finding was resolved by removing the `OptimisticOracleModule` in PR #290, where it was replaced by the `OOReporterModule`.

POLY1-09	Gas Griefing Of Batched `migratePositions` Calls		
Assets	`src/modules/NegRiskModule.sol`		
Status	**Resolved:** See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

A permissionless caller can resolve a migrated neg-risk condition ahead of its rightful holder, blocking subsequent `migratePositions()` calls and griefing batched migration transactions into reverting after partial work has been done.

In `NegRiskModule`, `resolveConditionToNo()` is permissionless, the outline of which can be seen below:

```
function resolveConditionToNo(ConditionId _conditionId) external {
    if (_isMigrationCondition(_conditionId)) {
        _resolveMigrationCondition(_conditionId);
        return;
    }
    ...
}
```

Once `_resolveMigrationCondition()` succeeds, the condition's result is stored:

`src/modules/migration/BaseMigrationMixin.sol::BaseMigrationMixin::_resolveMigrationCondition()`

```
uint256[] memory result_ = new uint256[](2);
result_[0] = legacyPayout0 * RESULT_DENOMINATOR / payoutDenominator;
result_[1] = RESULT_DENOMINATOR - result_[0];

_finalizeMigrationResolution(_conditionId, result_);
// ...
_redeemLegacyPositions(legacyConditionId_);
_settleLegacyCollateralToVault();
```

However, `migratePositions()` rejects any migrated condition that is already resolved:

`src/modules/migration/BaseMigrationMixin.sol::BaseMigrationMixin::_buildTransferAndMint()`

```
ConditionId conditionId = _legacyConditionIdToConditionId(legacyConditionId_);
require(_isMigrationCondition(conditionId), MigrationNotRegistered());
require(result[conditionId].length == 0, ConditionAlreadyResolved());
```

As a result, once any third party permissionlessly finalises a migrated neg-risk condition, holders of the corresponding legacy positions or the operator can no longer migrate those positions via `migratePositions()`.

This creates two related problems:

1. **Denial of late migration**
   A legacy holder may still own unresolved legacy positions and intend to migrate them after the legacy market has already resolved. But any third party can front-run or simply call `resolveConditionToNo()` first, causing future `migratePositions()` calls for that condition to revert with `ConditionAlreadyResolved()`.
2. **Gas griefing for batched migrations**
   `migratePositions()` processes arrays of legacy conditions in a single batch. If a user or operator submits a batch containing many valid legacy conditions, an attacker can front-run by resolving just one migrated condition which is in the end of the batch. When the victim transaction executes, the loop encounters the now-resolved condition and reverts the entire transaction, wasting gas for the caller.

This issue has a low impact as it does not cause loss of funds: legacy holders retain their underlying value and can still recover it through the resolved condition's payout. The harm is limited to wasted gas on griefed batches and forced abandonment of the migration path for affected conditions.

This issue has a low likelihood as the attack offers no direct economic gain to the griefer and requires precise front-running of migration batches or targeted resolution of conditions a specific user intends to migrate.

Recommendations

Prevent arbitrary callers from triggering `_resolveMigrationCondition()` through `resolveConditionToNo()` for migrated conditions.

Resolution

The finding has been resolved in commit 7356a3 by allowing legacy position redemption even when the condition is already resolved.

Condition resolution has been moved inside a conditional block called by `_redeemIfResolved()` that is only called when the condition has not yet been resolved.

The original callpoint `resolveConditionToNo()` has also been removed from the `NegRiskModule` file.

POLY1-10	Gas-Grief Of Operators By Invalidating Transfers Before Matching Orders		
Assets	`src/exchange/Exchange.sol`		
Status	**Closed:** See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The order matching flow validates order state, signature, fill amounts, and fee bounds before execution, but it does not guarantee that either side still has the necessary token balances and allowances at the moment the actual transfers occur.

As a result, either:

- A **maker** can transfer away required position tokens or reduce token approvals.
- A **taker** can transfer away required collateral or reduce allowance to the exchange.

Either will cause the match to revert only after meaningful gas has already been consumed processing the batch. This creates a practical gas-griefing vector against operators.

Regarding maker side:

In `_executeComplementaryBuyFastPath()`, the maker is validated first, but the maker's position transfer only happens afterwards:

`src/exchange/Exchange.sol::Exchange::_executeComplementaryBuyFastPath()`

```
totalFees = ctx.takerFee;

for (uint256 i; i < makerOrders.length; ++i) {
    Order calldata m = makerOrders[i];
    if (takerOrder.makerAmount * m.makerAmount < takerOrder.takerAmount * m.takerAmount) {
        revert NotCrossing();
    }

    {
        uint256 fill = fillAmounts[i];
        uint256 fee = makerFees[i];
        bytes32 mHash = _validateAndUpdate(m, fill, fee);
        uint256 taking = _calculateTakingAmount(fill, m.makerAmount, m.takerAmount);
        // @audit maker order is validated

        POSITION_MANAGER.unsafeTransferFrom(m.maker, ctx.takerAddr, m.tokenId, fill);
        // @audit transfer is attempted later, potentially after several iterations of the loop
        if (fee > taking) revert FeeExceedsProceeds();
        unchecked {
            COLLATERAL_TOKEN.safeTransferFrom(ctx.takerAddr, m.maker, taking - fee);
            totalFees += fee;
            takerMakingAmount += taking;
            takerTakingAmount += fill;
        }
        _emitOrderFilled(mHash, ctx.takerAddr, m, fill, taking, fee);
        if (fee > 0) _emitFeeCharged(FEE_RECEIVER, fee);
    }
}
```

So a maker can:

1. Sign a valid order,
2. be included in a batch,
3. then transfer away the required position balance before matching orders execution.

This causes `POSITION_MANAGER.unsafeTransferFrom()` to revert late in the loop.

Regarding taker side:

The taker can grief in the same transaction path by not maintaining enough collateral balance or allowance.

In the same complementary buy path, collateral is pulled from the taker inside the per-maker loop:

`src/exchange/Exchange.sol::Exchange::_executeComplementaryBuyFastPath()`

```
POSITION_MANAGER.unsafeTransferFrom(m.maker, ctx.takerAddr, m.tokenId, fill);
if (fee > taking) revert FeeExceedsProceeds();
unchecked {
    COLLATERAL_TOKEN.safeTransferFrom(ctx.takerAddr, m.maker, taking - fee);
    totalFees += fee;
    takerMakingAmount += taking;
    takerTakingAmount += fill;
}
```

And after the loop, additional taker-side collateral is also pulled to pay total fees:

`src/exchange/Exchange.sol::Exchange::_matchComplementaryOrders()`

```
if (totalFees > 0) {
    if (takerOrder.side == Side.BUY) {
        COLLATERAL_TOKEN.safeTransferFrom(ctx.takerAddr, FEE_RECEIVER, totalFees);
    } else {
        COLLATERAL_TOKEN.safeTransfer(FEE_RECEIVER, totalFees);
    }
}
```

So even if all maker-side transfers succeed, the transaction can still revert late if the taker no longer has enough collateral balance or has reduced allowance to the exchange.

This means a taker can intentionally let the batch progress and fail only during one of the collateral pulls, forcing the operator to absorb gas costs after substantial work has already been performed.

This can:

- Impose repeated gas costs on operators.
- Reduce batch matching reliability.
- Create denial-of-service pressure on offchain matching systems.

This issue has a low impact as it does not cause loss of user funds and only imposes wasted gas on the operator after the batch has already executed work that ultimately reverts.

This issue has a low likelihood as a griefer must repeatedly sign valid orders, get them included in a batch, then race to invalidate balances or allowances before matching, all without direct economic gain.

Recommendations

The matching flow should not rely on transfer-time failure as the first solvency check.

Possible mitigations include:

- Pre-check balances and allowances for both sides before entering expensive loops, especially for complementary fast paths.
- Using pull/escrow style order funding in which makers and/or takers commit assets ahead of execution.

Resolution

The development team have noted that this finding is mitigated by offchain checks.

POLY1-11	`OracleAggregator` Dispute Window Continues Running While Paused		
Assets	`src/oracle/OracleAggregator.sol`		
Status	**Closed:** See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The Oracle Aggregator's dispute window keeps elapsing while the contract is paused, allowing a proposal to become finalisable with little or no usable dispute time.

When a report reaches threshold, the aggregator stores the proposal hash and starts the dispute timer:

`src/oracle/OracleAggregator.sol::reportResult()`

```
if (newCount >= cfg.reporterThreshold && state.proposedResultHash == bytes32(0)) {
    state.proposedResultHash = resultHash;
    state.disputeWindowEnd = uint40(block.timestamp + cfg.livenessWindow);

    emit OutcomeProposed(_requestId, resultHash, _result);
}
```

However, reporting, disputing, finalising, and resolving are blocked while the aggregator is paused because the relevant functions are guarded by `whenUnpaused`:

`src/oracle/OracleAggregator.sol::reportResult()`

```
function reportResult(bytes32 _requestId, uint256[] calldata _result) external whenUnpaused {
```

`src/oracle/OracleAggregator.sol::disputeResult()`

```
function disputeResult(bytes32 _requestId) external whenUnpaused {
```

`src/oracle/OracleAggregator.sol::resolveResult()`

```
function resolveResult(bytes32 _requestId, uint256[] calldata _result) external whenUnpaused {
```

`src/oracle/OracleAggregator.sol::finalize()`

```
function finalize(bytes32 _requestId, uint256[] calldata _result) external whenUnpaused {
```

But the dispute deadline is a fixed timestamp based only on the original proposal time `state.disputeWindowEnd = block.timestamp + cfg.livenessWindow` and there is no code that adjusts it for pause duration.

Finalisation only checks that the stored deadline has passed:

`src/oracle/OracleAggregator.sol::finalize()`

```
ResolutionState storage state = resolutionStates[_requestId];
require(_isActiveStatus(state.status), RequestNotActive());
require(state.proposedResultHash != bytes32(0), ThresholdNotMet());
require(block.timestamp >= state.disputeWindowEnd, DisputeWindowActive());
require(keccak256(abi.encode(_result)) == state.proposedResultHash, InvalidResultHash());

_finalizeConditions(_requestId, cfg, _result);

emit RequestResolved(_requestId, state.proposedResultHash, address(this));
```

Therefore, if the protocol is paused during most or all of the dispute window, the deadline still expires in wall-clock time even though disputers were unable to act.

An example of this situation is such:

- `livenessWindow = 1 day`,
- a result reaches threshold at `T0`,
- `disputeWindowEnd = T0 + 1 day`.

Then:

1. At `T0 + 1 hour`, admin pauses the aggregator.
2. For the next 23 hours `disputeResult()` is impossible because `whenUnpaused` blocks it.
3. At `T0 + 25 hours`, the aggregator is unpaused.
4. Now `block.timestamp >= disputeWindowEnd` and so `finalize()` can be called immediately.

In practice, disputers had only 1 hour of usable dispute time, not the configured 1 day.

In the worst case, if the aggregator is paused almost immediately after proposal, disputers may have effectively zero meaningful opportunity to challenge the proposal before it becomes finalisable.

This issue has a low impact as it weakens the dispute liveness guarantee but does not directly cause loss of funds.

This issue has a low likelihood as it requires a pause to coincide with an active dispute window and a proposal that disputers would otherwise have challenged.

Recommendations

Modify the implementation such that the dispute deadline should account for pause duration, so that paused time does not consume disputed time.

Resolution

The development team have noted this is a design decision as paused markets are resolved directly or via the arbitrator, not unpaused.

POLY1-12	`OracleAggregator` Does Not Enforce Majority Reporter Quorum		
Assets	`src/oracle/OracleAggregator.sol`		
Status	**Closed:** See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

`OracleAggregator` allows a result to become the active proposal as soon as the number of matching reporter submissions reaches `reporterThreshold`. However, there is no requirement that `reporterThreshold` be greater than half of the registered reporter modules.

As a result, if `reporterThreshold` is configured below strict majority, a minority coalition of reporter modules can be the first to reach threshold and permanently set `proposedResultHash`, even if a majority of the registered reporter modules would have reported a different result.

The proposal is locked here once threshold is reached:

`src/oracle/OracleAggregator.sol::OracleAggregator::reportResult()`

```
state.proposedResultHash = resultHash;
state.disputeWindowEnd = uint40(block.timestamp + cfg.livenessWindow);

emit OutcomeProposed(_requestId, resultHash, _result);
```

This means:

- A minority coalition can preempt a later majority,
- the majority cannot replace the proposed result through normal reporting,
- and the only remaining recourse becomes dispute/arbitration.

After that point, the system transitions into dispute/arbitration flow rather than continuing to aggregate reporter votes toward a true majority-selected outcome. Therefore, the current implementation is threshold-first, not majority-quorum-based.

If the intended semantics are that reporter modules should collectively determine the proposed result by majority, then the current configuration model is incorrect unless it enforces:

- `reporterThreshold > reporterModules.length / 2`

For example, assume:

- There are 6 registered reporter modules.
- `reporterThreshold = 2`

Now suppose reporting occurs as follows:

1. Reporter A reports `payout = 0`

2. Reporter B reports `payout = 0`

At this point, the threshold of 2 is reached, so the aggregator stores `proposedResultHash = hash([0])` and opens the dispute window.

Now the remaining 4 reporters submit `payout = RESULT_DENOMINATOR`

So the final reporter distribution is:

- 2 reporters support `0`
- 4 reporters support `RESULT_DENOMINATOR`

Even though $4/6$ is the majority, the majority result is not the proposed outcome.
The minority result reached threshold first and locked in the proposal.

Therefore, the contract does not implement majority quorum correctly under such configurations.

This issue has a low impact as any incorrect minority-led proposal can still be challenged through the dispute and arbitration flow before it finalises, so it does not directly cause loss of funds.

This issue has a low likelihood as it depends on the operator configuring `reporterThreshold` below strict majority of the registered reporter modules, which may be intentionally done to achieve a different reporting model, albeit with weaker guarantees.

Recommendations

If the intended model is majority-based reporter consensus, enforce the following during request initialisation:

```
reporterThreshold > reporterModules.length / 2
```

Resolution

The development team have closed this issue with the following comment:

> "*Strict majority not required; a reporter conflicting with the stored proposal warrants a dispute — no use case for two concurrent proposals.*"

POLY1-13	`EOAReporterModule` Singleton Claim Is Inconsistent With Aggregator Voting Model
Assets	`src/oracle/modules/reporters/EOAReporterModule.sol` `src/oracle/OracleAggregator.sol`
Status	**Resolved:** See Resolution
Rating	Informational

Description

`EOAReporterModule` documents itself as a "Singleton reporter module for authorised EOA addresses" and seems to be intended as a way to route EOA reports to the Oracle Aggregator, ensuring at most one vote per EOA per event. However, the Oracle Aggregator only accepts one report per `msg.sender`, and thus only the first EOA's vote is counted. As a result, the module is either misleading or its use is fundamentally broken at the design level.

The module is initialised as a reporter for an event via the `initializeReporterModule()` function, wherein multiple reporters are authorised:

`src/oracle/modules/reporters/EOAReporterModule.sol::EOAReporterModule::initializeReporterModule()`

```
// Decode list of authorized reporters
address[] memory reporters = abi.decode(data, (address[]));

for (uint256 i = 0; i < reporters.length; i++) {
    authorizedReporters[eventId][reporters[i]] = true;
    emit ReporterAdded(eventId, reporters[i]);
}
```

EOAs submit to the oracle via the module's `report()` function, which validates the caller against `authorizedReporters` and ensures that each caller submits at most once per event via the mapping `hasReported`. It then calls the aggregator to forward the report:

`src/oracle/modules/reporters/EOAReporterModule.sol::EOAReporterModule::report()`

```
function report(bytes32 _conditionId, uint256[] calldata _result) external {
    ConditionId conditionId = ConditionIdLib.from(_conditionId);
    EventId eventId = conditionId.eventId();

    require(requestInitialized[eventId.asCondition()], RequestNotInitialized());
    require(authorizedReporters[eventId][msg.sender], NotAuthorizedReporter());
    require(!hasReported[conditionId][msg.sender], AlreadyReported());

    hasReported[conditionId][msg.sender] = true;

    emit Reported(_conditionId, msg.sender, keccak256(abi.encode(_result)));

    // Forward to aggregator
    IOracleAggregator(aggregator).reportResult(_conditionId, _result);
}
```

However, `OracleAggregator.reportResult()` then applies its `AlreadyVoted` guard against `msg.sender`, which is the module itself (not the original EOA):

src/oracle/OracleAggregator.sol::OracleAggregator::reportResult()

```
function reportResult(bytes32 _requestId, uint256[] calldata _result) external whenUnpaused {
    (EventId eventId, RequestConfig storage cfg) = _getRequestConfigForRequestId(_requestId);
    require(isReporterModule[eventId][msg.sender], NotRegisteredModule());
    require(!hasReporterVoted[_requestId][msg.sender], AlreadyVoted());
    // @audit msg.sender is the EOAReporterModule address, not the underlying EOA

    hasReporterVoted[_requestId][msg.sender] = true;

    // ... snip ...
}
```

The two readings produce contradictory expectations:

1. The module is genuinely a singleton (one deployment serves many requests, and `authorizedReporters` is populated with several EOAs per request as the structure suggests). In this case, the contract is broken in spirit: only the first authorised reporter to call `report()` for a given condition succeeds, and every subsequent authorised reporter reverts at the aggregator with `AlreadyVoted`. The `authorizedReporters` set, the per-reporter `hasReported` mapping, and the per-reporter `AlreadyReported` error all imply multi-reporter behaviour that the system cannot deliver.
2. The intended deployment model is one `EOAReporterModule` proxy per reporter (so each proxy contributes one vote, and `reporterThreshold > 1` is met by registering multiple module proxies under one request). This matches the behaviour validated by `test_revert_reportResult_alreadyVoted` and `test_reportResult_differentModulesSameRequest` in `OracleAggregator.t.sol`. In this case, the singleton claim in the contract's NatSpec is wrong and the per-reporter `hasReported` mapping is dead state.

Either way, the contract surface advertises behaviour that the integrated system does not provide. An operator reading only `EOAReporterModule.sol` is likely to assume the singleton model in (1), provision one shared proxy with several authorised EOAs, and find that reporter consensus cannot be reached because only one EOA's vote is ever counted.

This issue is rated as informational as the system still resolves correctly under the per-reporter-proxy (2) deployment model, no funds are at risk, and the behaviour is exercised by tests; the defect is in the contract's documentation rather than in its runtime behaviour.

Recommendations

Consider aligning the module surface with the actual voting model:

- If one module is intended to back at most one report, the singleton wording and the `hasReported` mapping should be tightened to reflect that.
- If one module is intended to back many reporters, the aggregator-side bookkeeping needs to identify the underlying reporter rather than the module address, so that each authorised EOA can contribute a distinct vote.

Resolution

The documentation was corrected in commit e80782 to reflect one-vote-per-module semantics.

POLY1-14	Incomplete Validation In Oracle Aggregator's `initializeRequest()`
Assets	`src/oracle/OracleAggregator.sol`
Status	**Closed:** See Resolution
Rating	Informational

Description

`OracleAggregator.initializeRequest()` is missing several checks that would catch obvious operator misconfiguration. Any misconfiguration is effectively permanent for that event, as reconfiguration or modification is not permitted.

The following validations are absent:

1. The `eventId`'s `resolutionChain()` is never compared against the chain the aggregator is deployed on. Downstream modules reject mismatched chains via `_enforceResolutionChain()`, so the aggregator will eventually fail to push a finalised result to the target. A misconfigured request wastes resources and generates log events which may confuse integrators.
2. `reporterThreshold` is only required to be `>= 1` and `disputerThreshold` is only required to be `>= 1` when at least one disputer is registered. Neither is bounded above by the number of registered modules, so an operator that registers fewer modules than intended can silently lock out reporting or dispute escalation.
3. The `reporterModules` is not required to contain at least one module. This condition would be caught if `reporterThreshold` were required to be `<= reporterModules.length`. Such a misconfiguration would require administrative intervention to the request.
4. `_registerReporterModules()` and `_registerDisputerModules()` accept duplicate `config.module` entries and `address(0)`. Duplicates re-set the registration mapping and re-call the module's initialiser, which can corrupt module-side state. The zero address is meaningless as a module.
5. When `disputerModules.length == 0`, `arbitratorModule` and `arbitratorInitData` are still accepted and stored. Arbitration cannot be triggered in this configuration, so any non-zero arbitrator is dead state at best and misleading at worst.
6. The encoded `eventId.arity()` is not validated against the module shape. Binary markets must have `eventId.arity() == 0`, while neg-risk markets must have `eventId.arity() >= 2`. Without these checks, a malicious or careless operator can submit an `eventId` with an arity that does not match the module, producing a request that cannot be resolved through the normal flow.

This issue is rated as informational as `initializeRequest()` is gated behind `onlyOperator` and the operator is a trusted role, so each gap depends on operator error rather than adversarial input. However, because the function cannot be called twice for the same `eventId`, any of these misconfigurations bricks resolution for that event and forces resolution via manual intervention by an admin.

Recommendations

Consider tightening `initializeRequest()` with the above validation to prevent misconfigurations.

Resolution

Points 3 and 5 have been resolved in commit e80782, but issues noted in points 1, 2, 4 and 6 remain present.

POLY1-15	Missing Upper Bound On `resultLength`
Assets	`src/oracle/OracleAggregator.sol`
Status	**Closed:** See Resolution
Rating	Informational

Description

An atomic neg-risk request created with a large `resultLength` can become permanently unresolvable, locking all positions in the market, because `_finalizeConditions()` must resolve every subcondition in a single transaction and can exceed the block gas limit.

When a request is initialised in `initializeRequest()`, the `resultLength` for an atomic neg-risk market is required to equal the event's arity, but no upper bound is enforced. The arity is a 16-bit field, and `NegRiskModule.getEventId()` permits any condition count in the range `[2, 65535]`, so an atomic neg-risk request may be registered with a `resultLength` of up to 65535.

src/oracle/OracleAggregator.sol::OracleAggregator::initializeRequest()

```
} else {
    // MarketType.ATOMIC_NEGRISK
    require(_params.resultLength == eventId.arity(), InvalidResultLength()); // @audit no upper bound on arity / resultLength
}
```

Both resolution paths, `finalize()` and `resolveResult()`, delegate to `_finalizeConditions()`, which iterates `resultLength` times. Each iteration performs an external call into the target's `reportResult()`, which for the neg-risk target writes a fresh length-2 dynamic array and updates aggregate state:

src/oracle/OracleAggregator.sol::OracleAggregator::_finalizeConditions()

```
for (uint256 i; i < resultLength; ++i) { // @audit unbounded single-transaction loop, must fit in one block
    // ... snip ...
    try IBinaryReporter(_cfg.targetContract).reportResult(reportConditionId, binaryResult) { }
    catch (bytes memory returnData) {
        // ... snip ...
    }
}
```

Each iteration costs approximately 75,000 to 90,000 gas, so finalisation stops fitting within a typical block gas limit at a few hundred conditions. As a result, the market can never be finalised and all positions minted against it become permanently locked.

This issue is rated as informational as `initializeRequest()` is restricted to the trusted operator role. Additionally, an atomic neg-risk market with several hundred subconditions is unlikely to occur under normal conditions.

Recommendations

Consider enforcing an explicit upper bound on `resultLength` for atomic neg-risk markets in `initializeRequest()`, chosen conservatively so that `_finalizeConditions()` always fits within the block gas limit with headroom for the per-condition target cost.

Resolution

The development team have acknowledged this issue and stated they will make use of offchain checks to prevent it becoming a problem.

POLY1-16	Oracle Request's Arbitrator Can Resolve Before Arbitration Is Triggered
Assets	`src/oracle/OracleAggregator.sol`
Status	**Closed:** See Resolution
Rating	Informational

Description

`OracleAggregator.resolveResult()` permits an arbitrator to finalise a request at any point in its lifecycle, bypassing the propose → dispute → arbitrate flow entirely. The function only checks that the caller is the arbitrator (or an admin) and that the request is not already resolved; it does not require the request to be in `ArbitrationRequested` status.

The function is the arbitrator's hook for delivering an arbitrated outcome and is expected to be invoked only after `disputeResult()` has escalated the request via `onArbitrationTriggered()`. However, the access check accepts any call from `cfg.arbitratorModule`:

src/oracle/OracleAggregator.sol::OracleAggregator::resolveResult()

```
function resolveResult(bytes32 _requestId, uint256[] calldata _result) external whenUnpaused {
    // ... snip ...
    ResolutionState storage state = resolutionStates[_requestId];
    if (state.status == ResolutionStatus.Resolved) return;
    // @audit rejects calls after resolution already reached

    bool isArb = msg.sender == cfg.arbitratorModule;
    bool isAdmin = hasAllRoles(msg.sender, _ROLE_0);
    require(isArb || isAdmin, NotAuthorized());
    // @audit no requirement that status == ArbitrationRequested when caller is arbitrator

    // ... snip ...

    emit RequestResolved(_requestId, resultHash, msg.sender);
}
```

There is no status guard on the arbitrator branch, so an arbitrator that calls `resolveResult()` while the request is still in `None` or `Active` status will write its chosen outcome and resolve the request immediately. Reporter consensus and the dispute procedure are skipped entirely.

This issue would have a reduced impact as the arbitrator is a trusted module set by the operator at request initialisation and already has full authority over the final outcome once arbitration is requested; the gap only lets it act before the typical flow has run, rather than granting it new authority.

As this issue requires either a malicious or compromised arbitrator and was reported to Polymarket in a previous audit it has been marked as informational only.

Recommendations

Consider gating the arbitrator branch on `ArbitrationRequested` status, while preserving the admin override.

Resolution

The development team are aware of this behaviour and provided the following comment:

> "*This override path is intentional and part of the trust model for emergency/admin resolution. It is not meant to require a prior arbitration-escalation state transition.* "

POLY1-17	Order Remaining Amount Truncated To 248 Bits
Assets	src/exchange/Exchange.sol src/exchange/OrderStructs.sol
Status	**Closed:** See Resolution
Rating	Informational

Description

An order's remaining fillable amount is packed into 248 bits with no bounds check, so an order whose `makerAmount` exceeds 2^{248} has its stored `remaining` silently truncated. When the truncated value collapses to zero, the order is treated as fresh and reset to its full `makerAmount`, allowing it to be filled repeatedly beyond the amount that was signed.

The `OrderStatus` struct stores the remaining fillable amount as a `uint248`, packed into a single slot alongside the `filled` flag:

src/exchange/OrderStructs.sol::OrderStatus

```
struct OrderStatus {
    /// @dev Whether the order has been fully filled
    bool filled;
    /// @dev Remaining fillable amount (packed with filled in one slot)
    uint248 remaining;
}
```

`_storeOrderStatus()` writes this slot in assembly, shifting `remaining` left by eight bits to make room for the `filled` flag in the low byte:

src/exchange/Exchange.sol::Exchange::_storeOrderStatus()

```
function _storeOrderStatus(bytes32 orderHash, uint256 remaining) internal {
    OrderStatus storage status = orderStatus[orderHash];

    assembly ("memory-safe") {
        sstore(status.slot, or(shl(8, remaining), iszero(remaining)))
        // @audit shl(8, remaining) discards the top 8 bits of `remaining`
    }
}
```

The `remaining` argument is a `uint256`, but `shl(8, remaining)` discards the top eight bits. The persisted remaining is therefore `remaining` mod 2^{248}. `remaining` is initialised from `order.makerAmount`, which is an unbounded `uint256` validated only against zero in `_validateOrder()`:

src/exchange/Exchange.sol::Exchange::_validateOrder()

```
if (order.makerAmount == 0 || fillAmount == 0) revert ZeroMakerAmount();
remaining = remaining == 0 ? order.makerAmount : remaining;
// @audit no check that order.makerAmount <= type(uint248).max
```

When `remaining` is a non-zero multiple of 2^{248} (for example 2^{248} or 2×2^{248}), `shl(8, remaining)` evaluates to `0` while `iszero(remaining)` is also `0` because `remaining` is non-zero. The entire slot is written as `0`. On the next fill, both `_validateOrder()` and `_updateOrderStatus()` read the stored `remaining` as `0` and reset it to the full `order.makerAmount`:

src/exchange/Exchange.sol::Exchange::_updateOrderStatus()

```
remaining = remaining == 0 ? order.makerAmount : remaining;
```

This reset can repeat on each subsequent fill, so the order is fillable for more than the `makerAmount` it was signed for, breaking the invariant that an order is filled at most up to its signed maker amount.

This issue is not realistically exploitable. Triggering the truncation requires `order.makerAmount` to be at least 2^{248} together with fills of comparable size. Each fill transfers the corresponding amount of collateral or outcome tokens, and no token in the system has a supply remotely approaching 2^{248}.

As this issue was reported to Polymarket in a previous audit it has been marked as informational only.

Recommendations

Consider validating that `order.makerAmount` does not exceed `type(uint248).max` in `_validateOrder()`, reverting otherwise.

Resolution

The development team are aware of this behaviour and provided the following comment:

> "*This only becomes relevant at impractically large maker amounts near the uint248 bound and is already screened by offchain order validation. We accept the packing tradeoff for realistic order sizes.* "

POLY1-18	Orders Have No Expiry
Assets	src/exchange/Exchange.sol src/exchange/OrderStructs.sol
Status	**Closed:** See Resolution
Rating	Informational

Description

Signed orders never expire. The `Order` struct carries a `timestamp` field that is included in the order hash, but the `Exchange` contract never compares it against `block.timestamp`, so a signature that crosses the order book remains fillable indefinitely until the order is fully filled or its maker pauses.

The `timestamp` field is documented in `OrderStructs.sol` as the moment the order was created, and it is part of the signed payload:

src/exchange/OrderStructs.sol::Order

```
struct Order {
    // ...
    /// @dev Unix timestamp at which the order was created
    uint256 timestamp;
    // ...
}
```

However, no code path reads or validates `order.timestamp`. The only validation performed on an order is in `_validateOrder()`, which checks fill status, user pause, maker amount, signature, and fee bounds, but performs no time-based check:

src/exchange/Exchange.sol::Exchange::_validateOrder

```
function _validateOrder(bytes32 orderHash, Order calldata order, uint256 fillAmount, uint256 fee)
    internal
    view
    returns (uint256 remaining)
{
    // ...
    if (isUserPaused(order.maker)) revert UserIsPaused();
    if (filled) revert OrderAlreadyFilled();
    if (order.makerAmount == 0 || fillAmount == 0) revert ZeroMakerAmount();
    remaining = remaining == 0 ? order.makerAmount : remaining;
    if (fillAmount > remaining) revert MakingGtRemaining();

    _validateSignature(orderHash, order);

    _validateFeeRate(fee, fillAmount, order);
}
```

As a consequence, the protocol relies entirely on the offchain operator backend to honour order freshness: makers sign orders that the backend is trusted to discard once stale, but the orders themselves never expire onchain. If a maker's signed-but-unfilled orders are retained (for example, leaked from, or replayed by, a compromised operator backend), they can be settled long after the maker considered them dead, at prices that may no longer reflect market conditions.

This issue is rated as informational as exploitation requires a compromised or malicious operator and a user who has changed their order intentions since signing.

Recommendations

Implement an order expiry mechanism that is enforced onchain.

Resolution

The development team have closed this finding stating this is intended behaviour as the operator backend honours order freshness.

POLY1-19	pUSD Lacks Fine-Grained User Controls For Compliance Responses
Assets	src/collateral/CollateralToken.sol
Status	**Closed:** See Resolution
Rating	Informational

Description

CollateralToken (pUSD) is a 1:1 wrapper over USDC and USDC.e but exposes no per-user administrative controls (no blocklist, no per-account freeze, no transfer hook), so the development team has no fine-grained mechanism for responding to a sanctions or court-order request that targets an individual user. The only kill switches available are coarse: per-asset pause on the on- and off-ramps.

USDC itself complies with OFAC sanctions and responds to US legal process: Circle has a documented history of freezing USDC balances at specific addresses on request. Because every pUSD token is backed 1:1 by USDC (or USDC.e) held in the external vault, pUSD effectively inherits this risk. Because the pUSD token does not expose any administrative controls to bring individual addresses into compliance, Circle may be compelled to freeze the vault in its entirety, effectively locking the entire system. If such controls did exist, Polymarket may be able to apply a more finer-grained response which is less disruptive.

This issue is exacerbated by the fact that pUSD has no pause / unpause functionality on transfers of pUSD : if the underlying vault is frozen by Circle, the restricted individual may continue to mix their pUSD tokens among several addresses, further frustrating any later efforts to form a targeted response. No controls let an administrator quarantine a single account, refuse a transfer, or seize and re-mint funds tied to an individual address.

This issue is rated as informational as the gap is a missing feature with regulatory and operational implications rather than an exploitable bug; the trade-off between censorship-resistance and compliance flexibility is a deliberate design choice, but the development team should make the choice consciously and document it.

Recommendations

Consider whether the pUSD token requires finer-grained compliance controls.

Resolution

The development team have closed this issue and will consider this issue more in the future.

POLY1-20	`removeBridge()` Can Strand In-Flight CCIP Bridge Transfers
Assets	`src/bridge/abstract/BridgeBase.sol` `src/modules/CombinatorialModule.sol`
Status	**Closed:** See Resolution
Rating	Informational

Description

The combinatorial module's `removeBridge()` flow can remove bridge connectivity while CCIP messages are still in flight, stranding those messages and making them permanently undeliverable.

`src/modules/CombinatorialModule.sol::CombinatorialModule::removeBridge()`

```
function removeBridge(address _bridge) external onlyAdmin {
    _removeRoles(_bridge, BRIDGE_ROLE);
}
```

If user Collateral or Position is already burned on the source chain before destination execution succeeds, removing the bridge during that in-flight period can make the pending destination execution permanently impossible, without any obvious generic recovery path.

This is especially dangerous for delayed-delivery cases, such as when the destination execution initially fails and must be retried manually later.

Even if a message is merely delayed and not lost, `OffRamp.execute()` validates execution against the current destination-side bridge configuration. So execution is not judged against the configuration that existed when the message was sent. It is judged against whatever bridge configuration exists at execution or retry time.

If `removeBridge()` removes the bridge while the message is still pending, this can cause the pending delivery to become permanently unexecutable.

The message path explicitly allows failed execution to remain retryable, which means delayed manual execution is an intended part of the system behaviour.

`OffRamp.execute()` permits retry from `UNTOUCHED` or `FAILURE`. This means a failed destination delivery is not terminal by itself; a later honest/manual execution attempt is expected to succeed if conditions remain valid. But if `removeBridge()` changes those conditions during the delay window, the retry path may disappear entirely.

A realistic example is:

1. User bridges collateral or a position.
2. Source-side burn succeeds.
3. The destination execution fails because the user set the gas limit too low.
4. The message remains retryable and waits for manual execution on the destination chain.
5. During the time that the message was pending to be retried, `removeBridge()` is called in the combinatorial module.
6. The destination bridge dependencies are no longer valid.
7. The delayed message can no longer be executed, even though the source-side asset was already consumed.

In that case:

- The source-side collateral or position is already gone.

- The destination-side collateral or position is never minted.
- The protocol does not provide a generic source refund or destination rescue mechanism.

Note that the same issue can happen for `removePeer()` and `removeModule()`.

In the worst case, an in-flight transfer can become permanently unexecutable, leaving the source-side asset consumed without a corresponding destination-side mint and no generic refund path. This is a significant user experience issue and can cause loss of funds for the affected transfer, but it does not cause systemic risk and is limited to the specific transfers that happen to be in flight during the action.

This issue requires an admin to invoke `removeBridge()`, `removePeer()`, or `removeModule()` while a CCIP message is in flight or pending manual retry, which can be avoided by pausing the sending side until messages are no longer pending.

As this issue was reported to Polymarket in a previous audit it has been marked as informational only.

Recommendations

Implement `removeBridge()`, `removePeer()`, and `removeModule()` such that they should not be able to invalidate unresolved in-flight transfers without a compensating recovery mechanism.

Resolution

The development team are aware of this issue and provided the following comment:

> "*Module replacement on bridged systems will only happen through an explicit migration plan that accounts for in-flight messages. This is an operational rollout constraint, not a user-triggerable runtime bug.* "

POLY1-21	Reserved Bits In Structured IDs Are Not Validated
Assets	src/libraries/Ids.sol
Status	**Closed:** See Resolution
Rating	Informational

Description

The structured ID encoding includes a 64-bit reserved region:

src/libraries/Ids.sol

```
// [moduleId(8) | baseHash(128) | arity(16) | reserved(64) | resolutionChain(16) |
//  conditionIndex(16) | outcomeIndex(8)]

uint256 constant RESERVED_BITS = 64;
```

The `from()` validation functions only check the bottom bits (outcome byte for `ConditionId`, bottom 24 bits for `EventId`) but do not validate that the reserved 64-bit region is zero:

src/libraries/Ids.sol::EventIdLib

```
function from(bytes32 _raw) internal pure returns (EventId) {
    if (uint256(_raw) & EVENT_SUFFIX_MASK != 0) revert NonCanonicalEventId(_raw);
    return EventId.wrap(bytes29(_raw));
}
```

src/libraries/Ids.sol::ConditionIdLib

```
function from(bytes32 _raw) internal pure returns (ConditionId) {
    if (uint256(_raw) & OUTCOME_MASK != 0) revert NonCanonicalConditionId(_raw);
    return ConditionId.wrap(bytes31(_raw));
}
```

This allows IDs with arbitrary values in the reserved region to be created and used throughout the system, for example creating positions via `split()`.

This issue is rated as informational as the reserved bits are currently unused, and there is currently no direct impact. However, if a future upgrade assigns meaning to the reserved bits and expects them to follow a particular structure, then tokens with non-zero reserved bits created under the current code may cause unexpected behaviour and forward compatibility issues.

Recommendations

Consider adding validation for the reserved bits in the `from()` functions to enforce canonicality. Alternatively, document the design decision to allow arbitrary reserved bits and plan for migration scenarios in future upgrades.

Resolution

The development team have closed this as a known issue.

POLY1-22	Miscellaneous General Comments
Assets	`/*`
Status	**Closed:** See Resolution
Rating	Informational

Description

This section details miscellaneous findings discovered by the testing team that do not have direct security implications:

1. **Inconsistent Admin Role Management Across Contracts**

   Related Asset(s): `src/auth/Roles.sol`, `src/auth/InitializableRoles.sol`, `src/oracle/mixins/Auth.sol`, `src/collateral/CollateralOnramp.sol`, `src/collateral/CollateralOfframp.sol`, `src/collateral/PermissionedRamp.sol`, `src/exchange/Exchange.sol` and `src/bridge/CcipBridge.sol`

   Authority over the admin role is assigned inconsistently across the contracts in scope, with three distinct patterns in use:

   (a) Asymmetric (owner adds, admin removes): `Roles.sol`, `InitializableRoles.sol`, and `oracle/mixins/Auth.sol` all expose `addAdmin()` gated on `onlyOwner` and `removeAdmin()` gated on `onlyAdmin`. This applies to the `PositionManager`, `BinaryModule`, `NegRiskModule`, `CombinatorialModule`, `OracleAggregator`, and the reporter modules that inherit `OracleModuleBase`.
   (b) Admin-managed (admins add and remove admins): `CollateralOnramp`, `CollateralOfframp`, and `PermissionedRamp` gate both `addAdmin()` and `removeAdmin()` on `onlyRoles(ADMIN_ROLE)`.
   (c) Owner-only (owner adds and removes): `Exchange` and `CcipBridge` gate both `addAdmin()` and `removeAdmin()` on `onlyOwner`.

   The asymmetric pattern in (1) is the easiest to misread: existing admins can unilaterally remove other admins (including the original deployment admin) without owner involvement, but cannot add new ones. The admin-managed pattern in (2) lets any admin both promote and demote arbitrary addresses, so a single compromised admin can replace the entire admin set without owner approval.

   This issue is rated as informational as the operator and admin roles are trusted by design and no privilege escalation crosses the owner boundary; the concern is consistency and the operational surprises that follow from contracts in the same system enforcing different rules for the same role.

   Consider standardising on a single pattern for admin role management across the codebase, and documenting the chosen model in the relevant role docs so that operators have a single mental model for who can add or remove admins.

2. **Payload Lengths Are Not Checked Against Expected Length**

   Related Asset(s): `src/oracle/modules/reporters/EOAReporterModule.sol` `src/oracle/modules/reporters/ChainlinkReporterModule.sol` `src/bridge/abstract/BridgeBase.sol` `src/bridge/CcipBridge.sol`

   Several entry points decode operator- or bridge-supplied payloads with `abi.decode()` without first asserting that the payload's encoded length matches the schema being decoded into. `abi.decode()` will revert when the input is too short, but it ignores any trailing bytes beyond what the target types describe. An encoded payload containing an incompatible schema but sufficient encoded length will silently decode into the current schema, possibly corrupting state or causing hard-to-diagnose misconfigurations.

   The most operator-facing instances are the reporter module initialisers, which are populated from the `initData` field of `OracleAggregator.initializeRequest()`:

- `EOAReporterModule.initializeReporterModule()` decodes `data` as `address[]`.
- `ChainlinkReporterModule.initializeReporterModule()` decodes `data` as `(bytes32, uint256, uint256)`.

Similar shapes appear in the bridge handlers in `BridgeBase` (`(address, BridgedPosition[])`, `(address, uint256)`, `(ConditionId, uint256[])`) and in `CcipBridge._ccipReceive()` (`address` from `_message.sender`).

If a future version of the system adds a field to one of these payloads and an operator pastes the new payload into a deployment still running the old code, the call will not revert: the old fields will be populated correctly and the new field will be silently ignored, leaving the request misconfigured in ways that may not surface until much later. The same risk applies if a payload is built against the wrong schema: the payload will decode without error, but the resulting decoded values will be incorrect and may cause misconfiguration or state corruption.

This issue is rated as informational as the affected paths are gated behind trusted roles (operator for reporter init, the bridge for cross-chain handlers) and the system as it stands today does not have multiple coexisting schema versions; the concern is forward-compatibility and operator confusion rather than an immediate exploit.

Consider checking the payload length against the expected encoded size before or immediately after calling `abi.decode()` at each site, so that a payload that does not match the schema is rejected explicitly.

3. **Source-Side Bridge Events Lack Message ID**

Related Asset(s): `src/bridge/abstract/BridgeBase.sol`

`PositionsBridged`, `CollateralBridged`, and `ResultBridged` are emitted on the source chain without the transport-layer message ID, so integrators cannot correlate a send-side log with the matching destination-side `*Received` event (which does carry `messageId`) without resorting to transport-specific log scraping.

This issue is rated as informational as the omission is a UX and observability concern for offchain indexers and not a runtime bug.

Consider adding the message ID returned by the underlying transport as an indexed field on each source-side bridge event so that integrators can pair send and receive logs directly.

4. **Redundant Storage Read**

Related Asset(s): `src/bridge/CcipBridge.sol`

The `_transportSend()` function reads the `peers[_dstChain]` storage slot twice within a single call, wasting gas on a redundant `SLOAD`.

`_transportSend()` first reads the slot to verify a peer is configured, then reads it again to populate the CCIP message `receiver` field:

src/bridge/CcipBridge.sol::CcipBridge::_transportSend()

```
function _transportSend(uint256 _dstChain, bytes memory _payload, bytes calldata _options) internal override {
    _validateChain(_dstChain);
    require(peers[_dstChain] != address(0), PeerNotSet()); // @audit first SLOAD

    Client.EVM2AnyMessage memory message = Client.EVM2AnyMessage({
        receiver: abi.encode(peers[_dstChain]), // @audit second SLOAD of the same slot
        data: _payload,
        tokenAmounts: new Client.EVMTokenAmount[](0),
        extraArgs: _options,
        feeToken: address(0)
    });

    IRouterClient(getRouter()).ccipSend{ value: msg.value }(uint64(_dstChain), message);
}
```

The second access is a warm `SLOAD` (~100 gas), so the redundancy costs roughly 100 gas per bridge send. The same pattern is present in `_bridgeQuote()`, which also reads `peers[_dstChain]` once for the validation `require` and again for the `receiver` field.

This issue is rated as miscellaneous as it has no security impact and only results in a minor, avoidable gas cost.

Cache `peers[_dstChain]` in a local variable in both `_transportSend()` and `_bridgeQuote()`, and use it for both the validation check and the `receiver` field.

5. **Position Tokens Do Not Adhere To ERC-1155 Specification**

   Related Asset(s): `src/positionManager/PositionManager.sol`

   The ERC-1155 tokens used to represent different market positions do not adhere to the ERC-1155 specification. The main missing feature is the `onERC1155Received` hook which is normally called on the `_to` address when sending ERC-1155 tokens to an address. This can lead to incomplete logic execution or transactions reverting if third parties integrate Polymarket position tokens assuming they match the specification.

   The ERC-1155 specification is outlined in EIP-1155, below is the relevant excerpt:
   "*"A contract MAY skip calling the ERC1155TokenReceiver hook function(s) if the mint operation is transferring the token(s) to itself. In all other cases the ERC1155TokenReceiver rules MUST be followed as appropriate for the implementation""*

   As the position tokens mint directly to users third party developers may expect these functions to call the `onERC1155Received` hook.

   Note that as this is a deliberate design mechanism by the Polymarket team this issue has been recorded as informational only. The issue only relates to potential third party integrations and causes no issues within Polymarket's codebase. While this specification deviation is documented in NatSpec comments of related functions, some developers may not research this deeply given the compliance with ERC-1155 specifications otherwise.

   Ensure that third parties are aware of this deviation from the ERC-1155 specification. Review external documentation to ensure this difference is highlighted in developer-aimed documentation.

6. **Stale Bridge Support Comments**

   Related Asset(s): `src/libraries/BridgePayloads.sol`

   The `BridgePayload` contract contains a reference to the now removed LayerZero bridge.

   ```
   // @dev Used by both LzBridge and CcipBridge.
   ```

   Including comments relating to outdated features may lead to confusion during future developments.

   Similar comments also exist in the following test files: `BridgeUnitTestBase.sol` and `BridgeTestBase.sol`

   Review and update code comments to reflect the current and intended future features.

7. **Signature Malleability Is Allowed**

   Related Asset(s): `src/exchange/Exchange.sol`

   The `_isValidEOASignature()` function accepts ECDSA signatures with a high `s` value, allowing signature malleability. For any valid signature `(r, s, v)` over an order hash, the complementary signature `(r, n - s, v')` (where `n` is the secp256k1 curve order) recovers the same signer and is also accepted, so a third party that observes a signed order can derive a second, distinct, but equally valid signature for the same order.

   The function recovers the signer with `ecrecover` after normalising `v`, but never constrains `s` to the lower half of the curve order:

   src/exchange/Exchange.sol::Exchange::_isValidEOASignature()

   ```
   function _isValidEOASignature(bytes32 hash, address maker, bytes calldata sig) internal pure returns (bool) {
     if (sig.length != 65) return false;

     uint8 v;
     bytes32 r;
     bytes32 s;

     assembly {
       r := calldataload(sig.offset)
       s := calldataload(add(sig.offset, 0x20))
       v := byte(0, calldataload(add(sig.offset, 0x40)))
     }

     if (v < 27) v += 27; // @audit no check that `s` is in the lower half of the curve order
     if (v != 27 && v != 28) return false;

     address recovered = ecrecover(hash, v, r, s);
     return recovered != address(0) && recovered == maker;
   }
   ```

The native `ecrecover` precompile does not reject high-`s` values, so the malleable counterpart is treated as a fully valid signature.

This issue is rated as informational. Order fill state and replay protection are keyed on the EIP-712 `orderHash` computed by `_computeStructHash()`, which hashes the order fields and deliberately excludes the `signature` bytes. As a result, a malleable second signature maps to the same `orderHash` and cannot be used to double-fill, replay, or otherwise bypass the `orderStatus` accounting onchain.

In addition to this this issue has previously been reported in another audit and so the Polymarket team are already aware and have outlined that offchain checks outside the audit scope prevent it from occurring.

Consider rejecting the complementary signatures by enforcing that `s` lies in the lower half of the secp256k1 curve order, mirroring the OpenZeppelin `ECDSA` implementation.

8. **Fallback Resolution Does Not Update Local Counters**

Related Asset(s): `src/modules/NegRiskModule.sol`

In `NegRiskModule._finalizeNegriskResolution()`, when the synthetic fallback condition is auto-resolved, the function updates the `conditionsResolved[eventId_]` and `resultsSum[eventId_]` storage slots but does not update the local copies `conditionsResolved_` and `resultsSum_`. The post-resolution sanity check immediately afterwards uses those stale locals:

src/modules/NegRiskModule.sol::NegRiskModule::_finalizeNegriskResolution()

```
if (conditionsResolved_ == conditionCount_ + 1) {
    require(resultsSum_ == RESULT_DENOMINATOR, InvalidResults());
}
```

On the call that creates the fallback, `conditionsResolved_` is still equal to `conditionCount_` rather than `conditionCount_ + 1`, so the check is silently skipped on that path.

This issue is rated as miscellaneous as the stored state remains consistent and there is no security impact, but the sanity check does not behave as a universal final invariant check on every path.

Update the local variables alongside the storage writes after the fallback is auto-resolved so the post-resolution check always runs when intended.

9. **Magic Number In IDs Library**

Related Asset(s): `src/libraries/Ids.sol`

In `Ids.isValidEventId()`, a hardcoded `232` is used as the bit shift amount rather than referencing one of the named constants declared at the top of the library:

src/libraries/Ids.sol::ConditionIdLib::isValidEventId()

```
function isValidEventId(ConditionId _id) internal pure returns (bool) {
    bool result;
    assembly {
        result := iszero(shl(232, _id))
    }
    return result;
}
```

This issue is rated as miscellaneous as it has no security impact today, but should the storage layout pattern change in the future this magic number could be overlooked while updating the library, leading to incorrect detection of event IDs.

Replace the literal with one of the existing named constants declared at the start of the library file, or introduce a new named constant in the same area, to reduce the maintenance overhead.

Recommendations

Ensure that the comments are understood and acknowledged, and consider implementing the suggestions above.

Resolution

The development team's responses to the raised issues above are as follows:

1. Acknowledged with no changes made.
2. Acknowledged, checks will be made offchain.
3. In commit 490a57 `messageId` was added to all listed events.
4. In commit 490a57 `peers[_dstChain]` is cached in memory rather than reading from storage twice.
5. Acknowledged, this is deliberate design and third-party developer documents will highlight this.
6. The stale comment was removed in commit 490a57.
7. Acknowledged, offchain checks will prevent this.
8. Acknowledged with the following comment "*Stored state remains consistent; the skipped post-resolution sanity check on the fallback path is accepted.*"

Appendix A Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurance. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

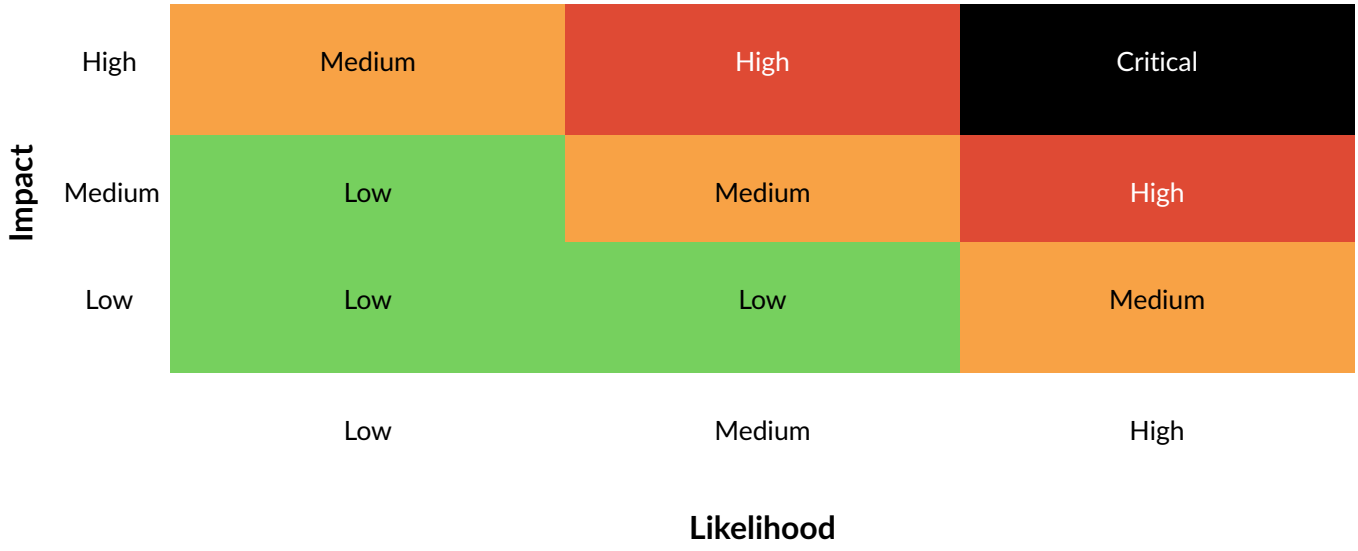

Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.